Siena Evers

April 1, 2025

CMPE 310 Project 2 Report

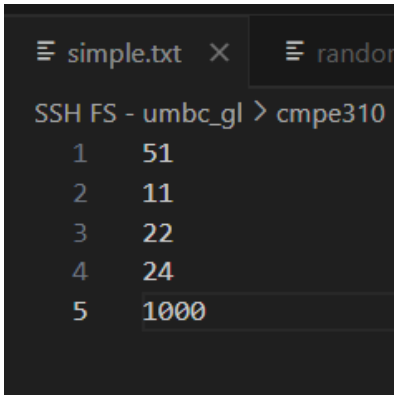
How I came up with this code

I started this project with pseudo code. I put comments in my code that broke down what I needed to accomplish in each section. The basic pseudo code included the variables/registers needed, the loops structure, and printing. Since using printf is allowed I used that for printing the results. Since we were taught to compile with elf64, I had to use the 64-bit registers for printf. I used 32-bit and below registers for the rest of the project though. Unlike the first project, I got my code to fully work this time.

Output of the Code

This was using randomInt100.txt (the file provided from Blackboard)

```
[severs1@linux6 ~/cmpe310]$ ./LabAssignment2
Program Finished, Sum is : 4679
```



The numbers for simple.txt is in the image in the left. This simple text file was used mostly during my time debugging the printing functionality.

```
[severs1@linux6 ~/cmpe310]$ ./LabAssignment2
Program Finished, Sum is : 1108
```

CODE:

```
;CMPE 310 Project 2
;Siena Evers, Spring 2025


;HOW TO COMPILE:
    ;nasm -g -f elf64 LabAssignment2.asm -o LabAssignment2.o
    ;gcc LabAssignment2.o -o LabAssignment2

section .data

    pathname db "randomInt100.txt"     ; randomInt100 ; simple.txt

    ;randomInt100 sum is 4679
    ;simple sum is 1108


    array dw 1000 dup(0) ;array, bits unknown

    ;using printf to print the sum
    msg db "Program Finished, Sum is : %d", 10, 0 ;msg
    len equ $-msg


section .bss
    buffer: resb 1024
    sum resb 1
    size resb 1    ;holds how many values are in the array


section .text

    global main
    extern printf


main:
```

```
    mov eax, 5
    mov ebx, pathname
    mov ecx, 0
    int 0x80
    jmp read


read:
    mov ebx, eax
    mov eax, 3
    mov ecx, buffer
    mov edx, 1024
    int 0x80


    ;Getting ready for the loop
    mov eax, 0  ;set eax to zero
    mov ecx, 0  ;i of the loop
    mov ebx, 0  ;counter for indexing array
    mov edx, 0  ;holds the number value

    jmp loop

loop:

    mov al, byte [buffer + ecx]

    cmp al, 0      ;if null loop stops
    je stopLoop1

    cmp al, 48     ;the char is below 48 which means it can't be a number
    jl increment

    cmp al, 57     ;the char is above 57 which means it can't be a number
    jg increment

   ;multiplying the existing number by 10 (more numbers with more than one
digit)
    mov edx, [array+ebx]
    imul edx, 10
```

```
    mov [array+ebx],edx

    sub al, '0' ; subtracting by the ascii value of 0

    ;add the digit to the array
    add [array+ebx], al

    ;increment through the file and loop again
    inc ecx
    jmp loop

;increments to the next spot on the array and increments through the text
file
increment:
    inc ecx
    inc ebx
    inc ebx
    jmp loop


stopLoop1:

    mov [size], ebx   ;edx is the max size of the array right now
    mov ebx, 0        ;set counter back to zero
    mov eax, 0        ;eax will be the sum
    mov edx, 0


loop2:

    cmp [size], ebx   ;if the size is less than the ebx, loop2 stops
    jl stopLoop2

    ;adds the element of the array into eax, and increment the index
    mov edx, [array+ebx]
    add eax, edx
    add ebx, 4

    jmp loop2
```

```
stopLoop2:
    mov [sum], eax      ;sets the sum


    ;printing with printf - using 64 registers because we use elf64 to
compile
    push rbp

    lea rdi, [rel msg]  ; address of msg
    mov rsi, [sum]      ; rsi has the sum value
    xor eax, eax
    call printf

    pop rbp ;pop off rbp

    jmp quit


quit:
    mov eax, 1
    mov ebx, 0
    int 0x80


;This is for debugging and making sure buffer is correct
print:
    mov edx, eax
    mov eax, 4
    mov ebx, 1
    mov ecx, buffer
    int 0x80
    jmp quit

;Old debugging code
print2:
    mov edx, len
    mov ecx, msg
    mov ebx, 1
```

```
    mov eax, 4
    int 0x80

    mov edx, 1
    mov ecx, sum
    mov ebx, 1
    mov eax, 4
    int 0x80
    jmp quit
```